RTOS GPIO
开发指南

版本号: 1.3
发布日期: 2025.07.01

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
1.0	2020.7.9	Allwinner	初始版本
1.1	2023.2.14	XAA0249	修改格式
1.2	2024.10.22	AWA1427	基于新模板更新文档
1.3	2025.07.01	AWA2298	增加平台描述

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

目　录

1 前言 **1**
1.1 文档简介 1
1.2 目标读者 1
1.3 适用范围 1
1.4 文档约定 2
1.4.1 标志说明 2
1.4.2 地址与数据描述方法约定 2
1.4.3 数值单位约定 2
1.5 相关术语介绍 3
1.5.1 硬件术语 3
1.5.2 软件术语 3

2 模块介绍 **4**
2.1 模块功能 4
2.2 模块配置 5
2.2.1 sys_config.fex 配置说明 5
2.2.2 menuconfig 配置说明 5
2.2.3 gpio_irq_map 配置说明 5
2.3 源码结构 6
2.4 软件框架 7

3 模块接口说明 **8**
3.1 数据结构 8
3.1.1 gpio_pin_t 8
3.1.2 gpio_driving_level_t 9
3.1.3 gpio_pull_status_t 9
3.1.4 gpio_data_t 9
3.1.5 gpio_power_mode_t 10
3.1.6 gpio_interrupt_mode_t 10
3.2 hal_gpio_check_valid 10
3.3 hal_gpio_get_data 10
3.4 hal_gpio_set_data 11
3.5 hal_gpio_set_direction 11
3.6 hal_gpio_get_direction 11
3.7 hal_gpio_set_pull 12
3.8 hal_gpio_get_pull 12
3.9 hal_gpio_set_driving_level 12
3.10 hal_gpio_get_driving_level 12
3.11 hal_gpio_pinmux_set_function 13

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.12 hal_gpio_pinmux_get_function 13
3.13 hal_gpio_sel_vol_mode 13
3.14 hal_gpio_set_debounce 14
3.15 hal_gpio_to_irq 14
3.16 hal_gpio_irq_request 14
3.17 hal_gpio_irq_free 15
3.18 hal_gpio_irq_enable 15
3.19 hal_gpio_irq_disable 15
3.20 hal_gpio_init 15

4 功能开发 17
4.1 功能概述 17
4.2 开发流程 17
4.2.1 配置 GPIO 为输入模式，并读取状态 17
4.2.2 配置 GPIO 为输出模式，并设置输出电平 17
4.2.3 配置 GPIO 为其它模块功能 PIN 18
4.2.4 配置中断模式 18
4.3 编程示例 18

5 调试方法 20
5.1 调试信息 20
5.2 调试工具 20
5.2.1 hal_gpio 20
5.2.2 hal_gpio_output 21
5.2.3 hal_gpio_input 21
5.2.4 hal_gpio_irq 21
5.2.5 hal_gpio_info 22

6 FAQ 23
6.1 一号通 FAQ 列表 23

版权所有 © 珠海全志科技股份有限公司。保留一切权利

表　格

表 1-1　适用产品列表 1
表 1-2　地址与数据描述方法约定 2
表 1-3　数值单位约定 2
表 1-4　硬件术语 3
表 1-5　软件术语 3
表 3-1　模块接口列表 8

广州星翼电子科技有限公司zdyzlinux

1 前言

1.1 文档简介

本文档介绍 AW 平台 RTOS 中 GPIO 驱动的接口及使用方法。

1.2 目标读者

- RTOS GPIO 模块开发工程师
- RTOS GPIO 模块使用工程师

1.3 适用范围

表 1-1: 适用产品列表

产品名称	内核版本	驱动文件
V459	Melis	hal_gpio.c
F133	Melis	hal_gpio.c
F135	Melis	hal_gpio.c
R328	FreeRTOS	hal_gpio.c
R329-DSP	FreeRTOS	hal_gpio.c
R128	FreeRTOS	hal_gpio.c
XR875	FreeRTOS	hal_gpio.c
MR527	FreeRTOS	hal_gpio.c
V821	FreeRTOS	hal_gpio.c

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1.4 文档约定

1.4.1 标志说明

注意

- **提醒操作中应注意的事项。不当的操作可能会损坏器件，影响可靠性、降低性能等。**

说明

为准确理解文中指令、正确实施操作而提供的补充或强调信息。

技巧

一些容易忽视的小功能、技巧。了解这些功能或技巧能帮助解决特定问题或者节省操作时间。

1.4.2 地址与数据描述方法约定

本文档在描述地址、数据时遵循如下约定：

表 1-2: 地址与数据描述方法约定

符号	例子	说明
0x	0x0200，0x79	地址或数据以 16 进制表示。
0b	0b010，0b00 000 111	数据采用二进制表示 (寄存器描述除外)。
X	00X，XX1	数据描述中，X 代表 0 或 1。 例如，00X 代表 000 或 001；XX1 代表 001，011，101 或 111。

1.4.3 数值单位约定

本文档在描述数据容量（如 NAND 容量）时，单位词头代表的是 1024 的倍数；描述频率、数据速率等时则代表的是 1000 的倍数。具体如下：

表 1-3: 数值单位约定

类型	符号	对应数值
数据容量（如 NAND 容量）	1 K	1024
	1 M	1 048 576
	1 G	1 073 741 824
频率，数据速率等	1 k	1000

版权所有 © 珠海全志科技股份有限公司。保留一切权利

类型	符号	对应数值
	1 M	1 000 000
	1 G	1 000 000 000

1.5 相关术语介绍

1.5.1 硬件术语

表 1-4: 硬件术语

术语	解释说明
MUX	multiplexer，数据选择器

1.5.2 软件术语

表 1-5: 软件术语

术语	解释说明
GPIO	General Purpose Input/Output，通用输入输出
HAL	Hardware Abstraction Layer，硬件抽象层

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2 模块介绍

2.1 模块功能

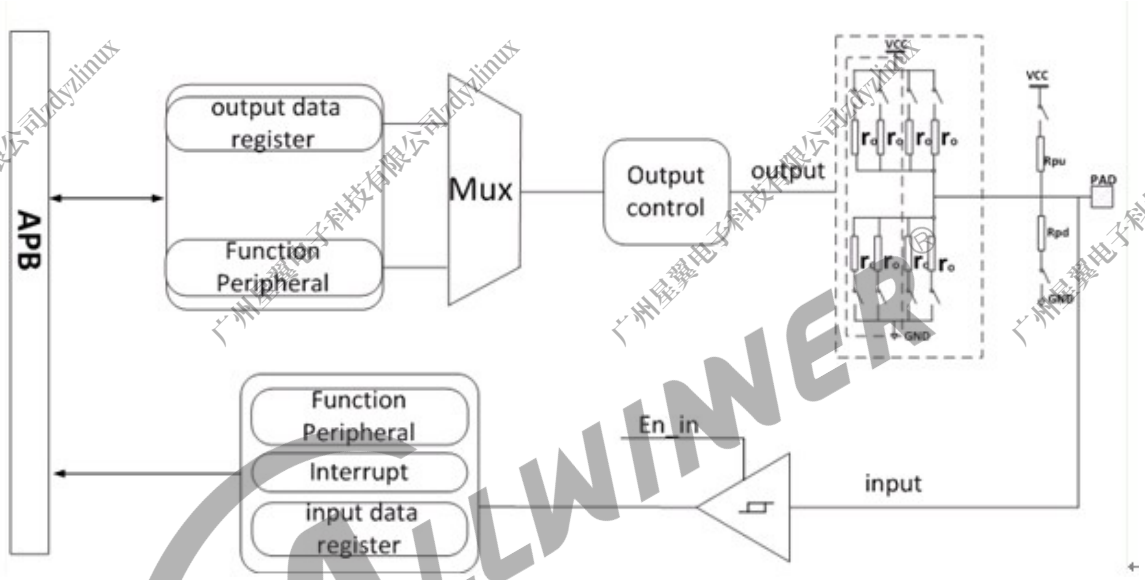

图 2-1: PORT 控制器

整个 GPIO 控制器由数字部分（GPIO 和外设接口）以及 IO 模拟部分（输出缓冲，双下拉，pad）组成。其中数字部分的输出可以通过 MUX 开关选择，模拟部分可以用来配置上下拉，驱动能力以及引脚输出电压等等。具体的规格如下：

- 可以在软件上配置各个引脚的状态
- 每个引脚都可以触发中断
- 可以配置上拉/下拉/无上下拉三种状态
- 每个引脚都可以配置 4 种驱动能力
- 可以配置边缘中断触发
- 中断数量跟随平台变化

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.2 模块配置

2.2.1 sys_config.fex 配置说明

其它模块若需要使用 GPIO，需要在 sys_config 中配置。

在描述 GPIO 的形式如下：

Port: 端口 + 组内序号

示例如下

```
uart_tx   = port:PA8<5><1><default><default>
```

2.2.2 menuconfig 配置说明

```
Drivers Options  --->
  soc related device drivers  --->
    [*] enable gpio driver   #使能GPIO驱动
    [ ]  enable gpio hal APIs test command   #GPIO驱动测试代码
    [ ] enable gpio chip aw9523   #支持GPIO拓展芯片
    [ ] enable amp gpio irq  #适用多个核都跑RTOS的平台，如R128/XR875
    [*] enable record gpio usage for debug #记录GPIO的使用情况，V821后才开始添加
```

说明

1. **“enable amp gpio irq” 选项适用多个核都跑 RTOS 的平台（如 R128/XR875），对应 CONFIG_DRIVERS_GPIO_SHARE_IRQ 配置，避免同一 port 的 GPIO 在多个核触发中断，。GPIO 中断的分配需要在 sys_config.fex 的 “gpio_irq_map” 配置。**
2. **Linux + RTOS 的 AMP 平台是 CONFIG_AMP_SHARE_IRQ 配置，未在 GPIO 驱动中配置，并且中断分配情况需要在 Linux 端配置。**

2.2.3 gpio_irq_map 配置说明

适用于 R128/XR875 平台等多个核都跑 RTOS 的平台。

（1）sys_config.fex 文件中 gpio_irq_map 主键，用于表示各个核使用 gpio 中断的情况；

（2）gpio_1_1、gpio_1_2 中每两个 bit 表示对应的 pin 的中断由哪个核负责处理。其中，0x0 表示不使用，0x1 表示 M33，0x2 表示 C906；

（3）比如 gpio_1_1 为 0xAAAAAAA，则表示 PA0~PA15 对应 gpio 中断由 c906 处理；

配置示例：

```
将PA19配置给M33使用（以便M33能使能其中断）

[gpio_irq_map]
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
gpio_1_1 = 0xAAAAAAAA
gpio_1_2 = 0xAAAAAA6A
gpio_2_1 = 0xAAAAAAAA
gpio_2_2 = 0xAAAAAAAA
gpio_3_1 = 0xAAAAAAAA
gpio_3_2 = 0xAAAAAAAA
```

2.3 源码结构

1. 驱动源码

```
hal/source/gpio/
├── aw9523.c
├── aw9523.h
├── gpio.h
├── hal_gpio.c
├── Kconfig
├── Makefile
├── sun20iw2
│   ├── gpio-sun20iw2.c
│   ├── Makefile
│   └── platform-gpio.h
├── sun300iw1
│   ├── gpio-sun300iw1.c
│   ├── Makefile
│   └── platform-gpio.h
├── sun50iw11
│   ├── gpio-sun50iw11.c
│   ├── Makefile
│   └── platform-gpio.h
├── sun55iw3
│   ├── gpio-sun55iw3.c
│   ├── Makefile
│   └── platform-gpio.h
....
....
```

2. 驱动 APIs 声明头文件

```
include/hal/
└── hal_gpio.h
```

3. 驱动 APIs 测试代码

```
hal/test/gpio/
├── Makefile
├── test_conf.h
└── test_gpio.c
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.4 软件框架

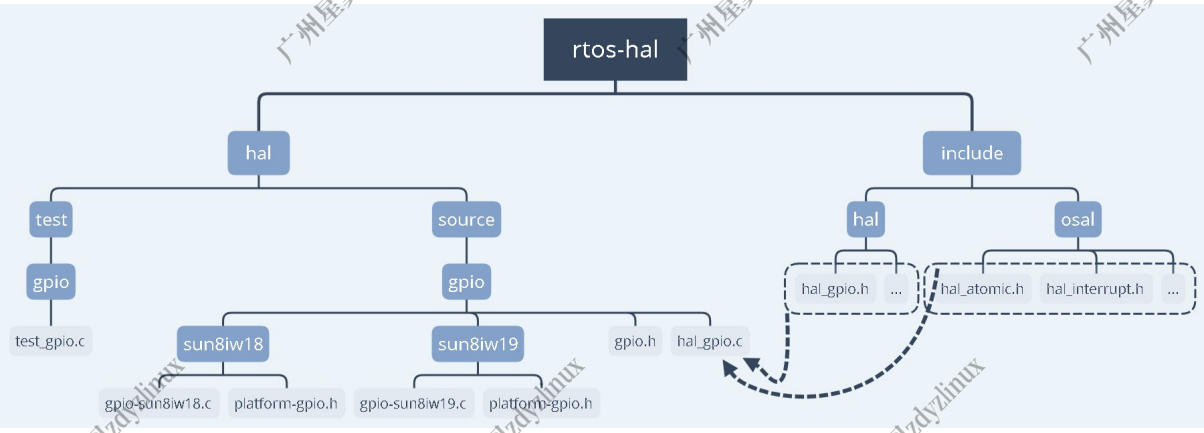

图 2-2: RTOS GPIO 驱动框架

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3 模块接口说明

表 3-1: 模块接口列表

API	解释说明
hal_gpio_check_valid	判断 GPIO 的合法性
hal_gpio_get_data	获取指定 GPIO 的电平状态
hal_gpio_set_data	设置指定 GPIO 的电平状态
hal_gpio_set_direction	设置指定 GPIO 的 IO 模式
hal_gpio_get_direction	获取指定 GPIO 的 IO 模式
hal_gpio_set_pull	设置指定 GPIO 的上下拉状态
hal_gpio_get_pull	获取指定 GPIO 的上下拉状态
hal_gpio_set_driving_level	设置指定 GPIO 的驱动能力
hal_gpio_get_driving_level	获取指定 GPIO 的驱动能力
hal_gpio_pinmux_set_function	设置指定 GPIO 的复用功能
hal_gpio_pinmux_get_function	获取指定 GPIO 的复用功能
hal_gpio_sel_vol_mode	设置指定 GPIO 组的电压模式
hal_gpio_set_debounce	设置指定 GPIO 组的中断采样频率
hal_gpio_to_irq	获取指定 GPIO 的 IRQ 中断号
hal_gpio_irq_request	GPIO 中断申请
hal_gpio_irq_free	GPIO 中断释放
hal_gpio_irq_enable	使能 GPIO 中断
hal_gpio_irq_disable	关闭 GPIO 中断
hal_gpio_init	GPIO 驱动初始化

3.1 数据结构

由于 GPIO 需要配置每个引脚的引脚复用功能，中断类型，驱动能力，上下拉，输出/输入数据，输入/输出方向等等，所以对 GPIO 的这些配置都封装在一个 enum 枚举结构里面，方便使用。下面是一些配置的定义，想要了解更多的数据结构可以到 hal_gpio.h 查看。

3.1.1 gpio_pin_t

该枚举定义了可用的每个引脚定义，在配置引脚的时候将相关参数传入则可，具体定义如下：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
typedef enum
{
    GPIO_PC0 = GPIOC(0),
    GPIO_PC1 = GPIOC(1),
    GPIO_PC2 = GPIOC(2),
    GPIO_PC3 = GPIOC(3),
    ...
    GPIO_PL0 = GPIOL(0),
    GPIO_PL1 = GPIOL(1),
    GPIO_PL2 = GPIOL(2),
    GPIO_PL3 = GPIOL(3),
    GPIO_PL4 = GPIOL(4),
    GPIO_PL5 = GPIOL(5),
} gpio_pin_t;
```

3.1.2 gpio_driving_level_t

该枚举定义了引脚的驱动能力的值，具体定义如下：

```
typedef enum
{
    GPIO_DRIVING_LEVEL0   = 0,      /**< Defines GPIO driving current as level0.  */
    GPIO_DRIVING_LEVEL1   = 1,      /**< Defines GPIO driving current as level1.  */
    GPIO_DRIVING_LEVEL2   = 2,      /**< Defines GPIO driving current as level2. */
    GPIO_DRIVING_LEVEL3   = 3       /**< Defines GPIO driving current as level3. */
} gpio_driving_level_t;
```

3.1.3 gpio_pull_status_t

该枚举定义了引脚的上下拉的值，具体定义如下：

```
typedef enum
{
    GPIO_PULL_DOWN_DISABLED   = 0,      /**< Defines GPIO pull up and pull down disable.  */
    GPIO_PULL_UP              = 1,      /**< Defines GPIO is pull up state.  */
    GPIO_PULL_DOWN            = 2,      /**< Defines GPIO is pull down state. */
} gpio_pull_status_t;
```

3.1.4 gpio_data_t

该枚举定义引脚的输入输出数据，具体定义如下：

```
typedef enum
{
    GPIO_DATA_LOW  = 0,             /**<  GPIO data low. */
    GPIO_DATA_HIGH = 1              /**<  GPIO data high. */
} gpio_data_t;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.1.5 gpio_power_mode_t

该枚举定义了引脚的电压模式，可以配置成 1.8V 和 3.3V，具体定义如下

```
typedef enum
{
   POWER_MODE_330 = 0,
   POWER_MODE_180 = 1
} gpio_power_mode_t;
```

3.1.6 gpio_interrupt_mode_t

该枚举定义了引脚的中断模式，具体定义如下：

```
typedef enum
{
   IRQ_TYPE_NONE      = 0x00000000,
   IRQ_TYPE_EDGE_RISING   = 0x00000001,
   IRQ_TYPE_EDGE_FALLING  = 0x00000002,
   IRQ_TYPE_EDGE_BOTH  = (IRQ_TYPE_EDGE_FALLING | IRQ_TYPE_EDGE_RISING),
   IRQ_TYPE_LEVEL_HIGH = 0x00000004,
   IRQ_TYPE_LEVEL_LOW  = 0x00000008,
} gpio_interrupt_mode_t;
```

3.2 hal_gpio_check_valid

- 作用：判断 GPIO 的合法性
- 参数：
  - pin：pin id
- 返回：
  - true: 合法
  - false: 不合法

3.3 hal_gpio_get_data

- 作用：获取指定 GPIO 的电平状态
- 参数：
  - pin:pin id
  - *data: 存放电平的指针变量
- 返回：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

  - -1: 失败
  - 0: 成功

3.4 hal_gpio_set_data

- 作用：设置指定 GPIO 的电平状态
- 参数：
  - pin:pin id
  - data: 需设置的电平高低
- 返回：
  - -1: 失败
  - 0: 成功

3.5 hal_gpio_set_direction

- 作用：设置指定 GPIO 的 IO
- 参数：
  - pin:pin id
  - direction: 需设置的 IO 模式
- 返回：
  - -1: 失败
  - 0: 成功

3.6 hal_gpio_get_direction

- 作用：获取指定 GPIO 的 IO 模式
- 参数：
  - pin:pin id
  - *direction: 存放 IO 模式的指针变量
- 返回：
  - -1: 失败
  - 0: 成功

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.7 hal_gpio_set_pull

- 作用：设置指定 GPIO 的上下拉状态
- 参数：
  - pin:pin id
  - pull: 需设置的上下拉状态
- 返回：
  - -1: 失败
  - 0: 成功

3.8 hal_gpio_get_pull

- 作用：获取指定 GPIO 的上下拉状态
- 参数：
  - pin:pin id
  - *pull: 存放上下拉状态的指针变量
- 返回：
  - -1: 失败
  - 0: 成功

3.9 hal_gpio_set_driving_level

- 作用：设置指定 GPIO 的驱动能力
- 参数：
  - pin:pin id
  - level: 需设置的驱动能力
- 返回：
  - -1: 失败
  - 0: 成功

3.10 hal_gpio_get_driving_level

- 作用：获取指定 GPIO 的驱动能力

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- 参数：
  - pin:pin id
  - *level: 存放驱动能力的指针变量
- 返回：
  - -1: 失败
  - 0: 成功

3.11 hal_gpio_pinmux_set_function

- 作用：获取指定 GPIO 的复用功能
- 参数：
  - pin:pin id
  - function_index: 需设置的复用功能
- 返回：
  - -1: 失败
  - 0: 成功

3.12 hal_gpio_pinmux_get_function

- 作用：设置指定 GPIO 的复用功能
- 参数：
  - pin:pin id
  - *function_index: 存放复用功能的指针变量
- 返回：
  - -1: 失败
  - 0: 成功

3.13 hal_gpio_sel_vol_mode

- 作用：设置指定 GPIO 组的电压模式
- 参数：
  - pin:pin id
  - pm_sel: 需设置的电压模式

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- 返回：
  - -1: 失败
  - 0: 成功

3.14 hal_gpio_set_debounce

- 作用：设置指定 GPIO 组的中断采样频率
- 参数：
  - pin:pin id
  - value: 需设置的值（bit0-clock select; bit6:4-clock pre-scale）
- 返回：
  - -1: 失败
  - 0: 成功

3.15 hal_gpio_to_irq

- 作用：获取指定 GPIO 的 IRQ 中断号
- 参数：
  - pin:pin id
  - *irq: 存放中断号的指针变量
- 返回：
  - -1: 失败
  - 0: 成功

3.16 hal_gpio_irq_request

- 作用：GPIO 中断申请
- 参数：
  - irq: 中断号
  - hdle: 中断处理函数
  - flags: 中断触发模式
  - *data: 私有数据指针变量
- 返回：
  - -1: 失败
  - 0: 成功

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.17 hal_gpio_irq_free

- 作用：GPIO 中断释放
- 参数：
  - 参数 1: 中断号
- 返回：
  - -1: 失败
  - irq: 成功释放的 irq 号

3.18 hal_gpio_irq_enable

- 作用：使能 GPIO 中断
- 参数：
  - irq: 中断号
- 返回：
  - -1: 失败
  - 0: 成功

3.19 hal_gpio_irq_disable

- 作用：关闭 GPIO 中断
- 参数：
  - irq: 中断号
- 返回：
  - -1: 失败
  - 0: 成功

3.20 hal_gpio_init

- 作用：GPIO 驱动初始化
- 参数：
  - void

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- 返回：
  - -1: 失败
  - 0: 成功

广州星翼电子科技有限公司zdyzlinux

4 功能开发

4.1 功能概述

GPIO 驱动提供了以下核心功能：

- 引脚配置：支持将 GPIO 引脚配置为输入、输出或特殊功能模式。
- 读取引脚状态：读取 GPIO 引脚的当前状态（高电平或低电平）。
- 设置引脚状态：设置 GPIO 引脚的状态（高电平或低电平）。
- 多路复用：支持将 GPIO 引脚配置为多种功能，如 I2C、SPI、UART 等通信接口。
- 中断配置：配置 GPIO 引脚为中断输入，并设置中断触发条件（上升沿、下降沿、双边沿）。

4.2 开发流程

4.2.1 配置 GPIO 为输入模式，并读取状态

步骤 1 调用 hal_gpio_pinmux_set_function 设置 GPIO_MUXSEL_IN。

步骤 2 调用 hal_gpio_set_direction 设置 GPIO_DIRECTION_INPUT。

步骤 3 调用 hal_gpio_set_pull 设置内部上下拉。

步骤 4 调用 hal_gpio_get_data 获取输入电平状态。

说明

步骤 1~3 没有严格顺序要求

4.2.2 配置 GPIO 为输出模式，并设置输出电平

步骤 1 调用 hal_gpio_pinmux_set_function 设置 GPIO_MUXSEL_OUT。

步骤 2 调用 hal_gpio_set_direction 设置 GPIO_DIRECTION_OUTPUT。

步骤 3 调用 hal_gpio_set_pull 设置内部上下拉。

步骤 4 调用 hal_gpio_set_driving_level 设置驱动能力。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

步骤 5 调用 hal_gpio_set_data 设置 GPIO_DATA_LOW 或 GPIO_DATA_HIGH。

说明

步骤 1~4 没有严格顺序要求

4.2.3 配置 GPIO 为其它模块功能 PIN

步骤 1 调用 hal_gpio_pinmux_set_function 设置具体的 GPIO_MUXSEL_FUNCTIONX。

步骤 2 调用 hal_gpio_set_pull 设置内部上下拉。

步骤 3 调用 hal_gpio_set_driving_level 设置驱动能力。

说明

步骤 1~3 没有严格顺序要求

4.2.4 配置中断模式

步骤 1 调用 hal_gpio_pinmux_set_function 设置为中断功能。

步骤 2 调用 hal_gpio_set_direction 设置 GPIO_DIRECTION_INPUT。

步骤 3 调用 hal_gpio_to_irq 获取相应 GPIO 的 IRQ 中断号。

步骤 4 调用 hal_gpio_irq_request 申请中断和注册中断处理函数。

步骤 5 调用 hal_gpio_irq_enable 使能 GPIO 中断。

说明

步骤 1~3 没有严格顺序要求

4.3 编程示例

1. 测试 demo 路径

lichee/rtos-hal/hal/test/gpio/test_gpio.c

2. 驱动调用示例

GPIO 一般在 sys_config 中配置，因此驱动需要先从 sysconfig 中获取 GPIO 信息。

如 UART 驱动所示：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
static void uart_pinctrl_init(uart_port_t uart_port)
{
#ifdef CONFIG_DRIVER_SYSCONFIG
    user_gpio_set_t gpio_cfg[4];
    int count, i;
    char uart_name[16];
    gpio_pin_t uart_pin[4];
    gpio_muxsel_t uart_muxsel[4];

    memset(gpio_cfg, 0, sizeof(gpio_cfg));
    sprintf(uart_name, "uart%d", uart_port);
    count = hal_cfg_get_gpiosec_keycount(uart_name);
    if (!count)
    {
        UART_ERR("[uart%d] not support in sys_config\n", uart_port);
        return ;
    }
    hal_cfg_get_gpiosec_data(uart_name, gpio_cfg, count);

    for (i = 0; i < count; i++)
    {
        uart_pin[i] = (gpio_cfg[i].port - 1) * 32 + gpio_cfg[i].port_num;
        uart_muxsel[i] = gpio_cfg[i].mul_sel;

        hal_gpio_pinmux_set_function(uart_pin[i], uart_muxsel[i]);
        hal_gpio_set_pull(uart_pin[i], GPIO_PULL_UP);
    }
#endif
}
```

1. hal_cfg_get_gpiosec_keycount 获取 uartx 节点 GPIO 配置数目。
2. hal_cfg_get_gpiosec_data 获取所有 GPIO 配置信息保存到 gpio_cfg 数组。
3. 遍历 gpio_cfg 数组保存的配置并逐一调用 GPIO 驱动接口设置。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5 调试方法

5.1 调试信息

GPIO 驱动默认不输出 DEBUG 信息，若需要确认 API 调用情况，需要配置 CONFIG_DRIVERS_GPIO_DEBUG。

路径：lichee/rtos-hal/include/hal/hal_gpio.h

```
#define CONFIG_DRIVERS_GPIO_DEBUG
#ifdef CONFIG_DRIVERS_GPIO_DEBUG
#define GPIO_INFO(fmt, arg...) hal_log_info(fmt, ##arg)
#else
#define GPIO_INFO(fmt, arg...) do {}while(0)
#endif
```

5.2 调试工具

- 配置 CONFIG_HAL_TEST_GPIO 后，可以使用 hal_gpio/hal_gpio_output/hal_gpio_input/hal_gpio_irq 等命令。
- 配置 ONFIG_DRIVERS_GPIO_RECORD_USAGE 后，可以使用 hal_gpio_info 命令。

5.2.1 hal_gpio

- 使用方法：hal_gpio
- 用途：基础 API 测试 Demo

使用示例：

```
cpu0>hal_gpio
Original: pin: 1 pull state: 0, dir: 15, data: 0x0, function_index: 15
Setting: pin: 1 pull state: 1, dir: 1, data: 0x1, function_index: 1
Results: pin: 1 pull state: 1, dir: 1, data: 0x1, function_index: 1
Test hal_gpio_set_pull API success!
Test hal_gpio_set_direction API success!
Test hal_gpio_set_data API success!
Test hal_gpio_pinmux_set_function API success!
Test hal_gpio_get_pull API success!
Test hal_gpio_get_direction API success!
Test hal_gpio_get_data API success!
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
Test hal_gpio_pinmux_get_function API success!
Test hal_gpio_to_irq API success!
Test hal_gpio_irq_request API success!
Test hal_gpio_irq_enable API success!
Test hal_gpio_irq_disable API success!
Test hal_gpio_irq_free API success!
Test gpio hal APIs success!
```

5.2.2 hal_gpio_output

- 使用方法：hal_gpio_output PXX value
- 用途：GPIO 输出控制测试

5.2.3 hal_gpio_input

- 使用方法：hal_gpio_input PXX
- 用途：GPIO 输入状态读取测试

使用示例：

```
uart>hal_gpio_intput PA3
Will read PA3 status for 10 seconds
Read gpio data test, [PA3]: 1
Read gpio data test, [PA3]: 1
Read gpio data test, [PA3]: 1
```

5.2.4 hal_gpio_irq

- 使用方法：hal_gpio_irq enable PXX
- 用途：GPIO 中断测试

使用示例：

```
uart>hal_gpio_irq enable PA03
pin: 3
gpio 3 to irq 203 succeed!
Test hal_gpio_to_irq API success!
[hal_gpio_irq_request]gpio_desc address is 0x42000000.
request irq 203 succeed!
Test hal_gpio_irq_request API success!
[hal_gpio_irq_enable]gpio_desc address is 0x42000000.
Test hal_gpio_irq_enable API success!
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5.2.5 hal_gpio_info

- 使用方法：hal_gpio_info
- 用途：GPIO 中断测试

使用示例：查看当前 GPIO 注册使用情况

```
uart>hal_gpio_info
====== GPIO information in use ======
[PC04]: mul_sel:4, pull:1, drv:0, irq:0
[PC05]: mul_sel:4, pull:1, drv:0, irq:0
uart>
```

说明

在 V821 平台上才开始添加，旧平台上不一定支持

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6 FAQ

6.1 一号通 FAQ 列表

暂无

广州星翼电子科技有限公司zdyzlinux

著作权声明

版权所有 ©2025 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

ALLWINNER、ALLWINNER 全志科技、全志科技、ALLWINNER（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。